

DOCUMENTO DE TRABAJO — TITULACIÓN

Ideas de proyecto de tesis

*Maestría en Sistemas Computacionales — panorama completo de opciones
evaluadas antes de elegir tema*

Juan Carlos Soto García

Instituto Tecnológico de Zacatecas — División de Estudios de Posgrado e Investigación

Septiembre 2026

CONTEXTO

Cómo elegir el tema

Documento de trabajo para elegir tema de tesis y solicitar la prórroga de titulación. Las ideas parten de trabajo que ya se hizo (no de cero), porque dado que ya se necesita prórroga, la variable que más importa ahora es **velocidad a defensa**, no la ambición del proyecto. Un tema que documenta y evalúa formalmente algo que ya se construyó y que ya corre en producción es más rápido de defender que uno nuevo: la ingeniería ya está hecha — falta la instrumentación, la evaluación y la redacción.

Cada idea trae: el problema, la propuesta, la contribución académica, por qué encaja con el perfil del candidato, qué tan rápido es, y qué riesgos tiene.

Recomendación rápida

#	IDEA	VELOCIDAD A DEFENSA	QUÉ TAN "DE CERO" ES
1	Framework comparativo de multi-tenancy en SaaS	Alta	Nada — 3 implementaciones ya existen
2	Gate de análisis estático para migraciones async seguras	Alta	Poco — la herramienta ya corre en CI
3	Skill + infraestructura para automatizar CR/pruebas a partir de HUs y prototipos (multi-repo)	Media	Parcial — la base ya existe, falta generalizarla
4	Catálogo de fallos y mitigaciones en sistemas de actores con estado	Media	Parcial — parte de un incidente real ya diagnosticado
5	FinOps automatizado para microservicios en Cloud Run	Media-baja	Sí — la automatización no existe todavía

Idea elegida: la #3, en su versión generalizada (agnóstica al sistema, alimentada por historias de usuario y prototipos). Es más trabajo que la #1, pero ya no depende de autorización de un tercero y es el aporte más original de la lista — queda como plan principal. Se desarrolla en detalle en el documento complementario *"Propuesta de tesis — idea elegida"*.

Framework comparativo de estrategias de multi-tenancy en SaaS

Problema. No existe un marco empírico estandarizado para elegir estrategia de multi-tenancy en una plataforma SaaS nueva; la decisión suele tomarse por intuición o por lo que el equipo ya conoce, sin comparar aislamiento, rendimiento, costo y complejidad operativa bajo condiciones controladas.

Propuesta. Ya se implementaron tres estrategias distintas en proyectos reales: discriminador `tenant_id` por documento en MongoDB, `schema-per-tenant` con `search_path` dinámico en PostgreSQL, y subcolección por tenant en Firestore. La tesis instrumenta las tres bajo una carga de trabajo comparable y mide: aislamiento de datos entre tenants (pruebas de fuga), latencia bajo carga concurrente, costo de infraestructura por tenant, y complejidad de migraciones de esquema.

Contribución académica. Un marco de decisión reproducible con datos reales (no solo la comparación teórica que ya existe en la literatura), más una matriz de "qué estrategia usar según N tenants esperados, sensibilidad de los datos y frecuencia de cambios de esquema".

Por qué encaja. Pocas personas pueden comparar las tres estrategias con implementaciones propias reales en vez de prototipos de laboratorio — la mayoría de la literatura compara teóricamente o con un solo caso.

Riesgos. Los tres proyectos no comparten carga de trabajo real idéntica, así que se necesita un benchmark sintético común para que la comparación sea justa (esto sí es trabajo nuevo, pero acotado: un script de carga, no un sistema).

Gate de análisis estático para migraciones asíncronas seguras

Problema. Migrar un codebase Python de llamadas síncronas a asíncronas (p. ej. a un driver nativo async de base de datos) es una fuente común de regresiones silenciosas: código que compila y corre, pero que le faltó un `await` y produce una coroutine sin ejecutar en vez de un error visible.

Propuesta. Ya se construyó exactamente esto: un gate de CI que bloquea llamadas async sin `await`, aterrizado en el mismo commit que una migración real a driver nativo async, con la suite completa en verde después. La tesis generaliza esa herramienta a un analizador estático reusable (más allá del caso puntual), y evalúa su efectividad reproduciendo regresiones históricas conocidas para medir tasa de detección y falsos positivos.

Contribución académica. Una herramienta open-source publicable (aporta algo tangible más allá del documento) + evidencia empírica de que el gate efectivamente habría bloqueado errores reales que ocurrieron antes de que existiera.

Por qué encaja. Ya existe el caso de uso real, el commit de referencia, y la suite de 2,741 pruebas como terreno de validación.

Riesgos. Es la idea más "técnica y acotada" de la lista — más fácil de terminar, pero un comité que busque algo más ambicioso puede pedir ampliarla (p. ej. a otros patrones de regresión, no solo `await` faltante).

Skill + infraestructura para automatizar revisión de código y pruebas a partir de HUs y prototipos

ELEGIDA

Versión final, generalizada. Se revisó y se cambió de "herramienta interna atada a una sola plataforma" a "framework agnóstico al sistema": en vez de evaluarse solo contra una plataforma actual, se diseña desde el inicio para operar sobre **cualquier repositorio o conjunto de repositorios**. Esto además resuelve de raíz el riesgo de autorización que tenía la versión anterior de esta idea: no depende de exponer métricas de un empleador, porque el objeto de estudio es la herramienta misma, no los datos de una plataforma particular.

Problema. Las herramientas de revisión de código y generación de pruebas asistidas por LLM que existen hoy (comerciales o internas de un equipo) suelen operar solo sobre el diff: no saben qué se pretendía construir. Revisan estilo y patrones genéricos, y generan pruebas que cubren líneas de código pero no verifican si se cumplió el comportamiento que pidió el negocio. Falta una capa de contexto explícita — la historia de usuario y el prototipo — que conecte "qué se quería" con "qué se construyó".

Propuesta. Diseñar e implementar una infraestructura y un *skill* (agente especializado, siguiendo el patrón de Claude Code Skills u otro framework de agentes) que:

1. **Indexa** uno o más repositorios: código, estructura, convenciones, historial de cambios.
2. **Ingiera como fuente de verdad** las historias de usuario (criterios de aceptación) y los prototipos/diseños (mockups, contratos de API) asociados a un cambio.
3. **Correlaciona código ↔ intención:** a partir de eso, genera revisión de código dirigida (no genérica) y pruebas derivadas explícitamente de los criterios de aceptación, no solo de cobertura de líneas.
4. Se valida en **al menos 2-3 sistemas distintos y no relacionados entre sí** (idealmente: uno o dos proyectos open source o propios ya públicos, y opcionalmente uno de un empleador si hay autorización) para demostrar que el enfoque generaliza y no es un script atado a un solo dominio.

Contribución académica. Un marco reproducible + una herramienta funcional que demuestra empíricamente que alimentar a un LLM con HUs y prototipos como contexto explícito mejora la relevancia de la revisión de código y la cobertura semántica de las pruebas generadas, frente a un enfoque que solo ve el diff. Métricas de evaluación sugeridas:

- **Precisión de revisión:** % de comentarios generados que un revisor humano califica como relevantes (vs. ruido).
- **Cobertura de criterios de aceptación:** % de criterios de la HU efectivamente verificados por las pruebas generadas (no solo % de líneas cubiertas).
- **Generalización:** la misma herramienta, sin reconfiguración manual profunda, corriendo sobre sistemas con stacks distintos.

Por qué encaja. Ya se construyó una versión atada a un solo sistema (el tablero de MRs y la generación de pruebas de humo) — se sabe qué funciona y qué no en la práctica. La generalización es el trabajo de tesis: extraer el patrón reusable de lo que ya se probó que funciona en un caso concreto.

Riesgos. Es la idea con más trabajo de ingeniería nuevo de las tres primeras (hay que construir la capa de indexación multi-repo y el mecanismo de ingesta de HUs/prototipos), pero a cambio es la más original, la más defendible como aporte propio, y ya no depende de autorización de terceros para el resultado principal — la experiencia en producción puede mencionarse como motivación sin necesitar exponer sus métricas.

IDEA 4

Catálogo de fallos y mitigaciones en sistemas de actores con estado

Problema. Los sistemas basados en el modelo de actores con persistencia de eventos (event sourcing) prometen resiliencia y escalabilidad horizontal, pero son propensos a fallos de consistencia distribuida difíciles de diagnosticar — la documentación de patrones de fallo reales (no teóricos) es escasa.

Propuesta. Se parte de un incidente real ya diagnosticado: un caso de *split-brain* en persistencia de eventos, causado por dos instancias no coordinadas escribiendo al mismo actor. La tesis generaliza ese diagnóstico en un catálogo de patrones de fallo en sistemas de actores con estado (cluster sharding, event sourcing), con pruebas de *chaos engineering* que reproducen cada patrón de forma controlada y validan la mitigación.

Contribución académica. Un catálogo validado empíricamente de fallos y mitigaciones — material directamente reusable por otros equipos que adopten el modelo de actores.

Por qué encaja. Ya existe un caso real completo (síntoma → diagnóstico → causa raíz → mitigación) que sirve de semilla; el resto es generalizar y probar sistemáticamente.

Riesgos. Requiere diseñar experimentos de fallo controlado (más trabajo nuevo que las ideas #1 y #2), y el proyecto de origen es de un cliente externo — mismo tema de autorización que tenía la idea #3 antes de generalizarse.

FinOps automatizado para microservicios en Cloud Run vía IaC

Problema. El ajuste de instancias mínimas/máximas y del dimensionamiento de servicios serverless suele hacerse manualmente y de forma reactiva, dejando dinero sobre la mesa o arriesgando disponibilidad.

Propuesta. Un sistema que, a partir de métricas reales de uso (Cloud Monitoring), recomienda y aplica ajustes de configuración vía Terraform de forma semi-automatizada, con un ciclo de retroalimentación medible en costo.

Contribución académica. Automatización del ciclo FinOps aplicado a IaC declarativo, con casos de ahorro medible como evidencia.

Por qué encaja. Esta optimización ya se hizo manualmente en producción — se conocen bien qué palancas mueven el costo.

Riesgos. Es la que más desarrollo nuevo requiere de las cinco — más lenta de terminar dado que ya se necesita prórroga.

SIGUIENTE PASO

Siguiente paso sugerido

1. ~~Resolver el tema de autorización de datos con el empleador~~ — ya no es un bloqueante duro para la #3 en su versión generalizada, porque el sistema se valida sobre repos propios/públicos.
2. Definir los 2-3 repositorios de validación (candidatos: proyectos propios ya públicos en GitHub, más opcionalmente uno de trabajo si hay autorización).
3. Redactar planteamiento del problema, objetivos general y específicos, y cronograma tentativo para el formato de prórroga — desarrollado en el documento complementario.